AMD AI DEVMASTER HACKATHON · TRACK 1 · MULTIMODAL AI · PROJECT PROFILE

Katha 🎙️📖

Speak a story idea — in English, Hindi, or six other languages — and get back a narrated, illustrated storybook video. Every model runs locally on an AMD Radeon GPU.

APPLICATION	**Katha** — voice-to-storybook multimodal creation tool
TEAM	**N DIVIJ** (solo developer)
HARDWARE	AMD Radeon PRO W7900 · gfx1100 (RDNA 3) · 48 GB VRAM · Radeon Cloud
STACK	ROCm 7.2.1 · PyTorch 2.9.1 (HIP) · Transformers · Diffusers · ffmpeg · Gradio
MODELS	Whisper large-v3-turbo · Qwen2.5-7B-Instruct · SDXL + IP-Adapter · Kokoro-82M
SOURCE	github.com/N-45div/Katha · MIT license

2.6 min
IDEA → FINISHED STORYBOOK

14.3 GB
PEAK VRAM — FITS 16 GB RADEONS

5
MODELS CHAINED, ALL LOCAL

0
CLOUD API CALLS



Frames from the first validated end-to-end run on the Radeon PRO W7900 — "*The Brave Dragon*", from the idea "a dragon who is afraid of fire" to a narrated video in one command. Subtitles are burned by the pipeline itself; the same hero, Finn, holds across every page. The run's `benchmark.json` is committed in the repository.

1 Project Background

Every parent knows the request: “one more story.” Every child has an idea — a rabbit astronaut, an elephant who wants to fly — that deserves better than a tired improvisation at 9 pm.

Generative AI can obviously help, yet the tools that exist share three structural problems that Katha treats as its design brief:

Typed prompts exclude the actual audience

Story tools are prompt boxes. A five-year-old cannot type a prompt — but they can *say* one, vividly, in whatever language they dream in. If the input is a keyboard, the child is a spectator. Katha is voice-first: the microphone is the product's front door.

The storytelling internet speaks English

Most story generators write and narrate only in English. A child in Jaipur or São Paulo gets a story in a language that isn't the one their grandmother tells stories in. Katha detects the spoken language with Whisper and keeps the entire experience — story text and narration voice — in that language, across eight languages including Hindi (*katha*, कथा, means “story”).

Children's voices do not belong in the cloud

A bedtime-story tool records children speaking and invents stories about them. That data is exactly what should never leave the house. Katha runs all five of its models — speech recognition, story writing, illustration, narration — on a local AMD Radeon GPU. Nothing is uploaded; there is no account, no subscription, no telemetry.

> **Why this is a Radeon showcase.** Katha is a deliberately maximal multimodal pipeline — audio in; text, image, and audio generation in the middle; video and print artifacts out — engineered so the whole chain fits and performs on one Radeon GPU through sequential model orchestration. It demonstrates that ROCm runs today's open-model stack (Whisper, Qwen, SDXL + IP-Adapter, Kokoro) end-to-end, at interactive speeds.

2 Target Users & Application Scenarios

USER	SCENARIO	WHAT KATHA PROVIDES
Parents & children	Bedtime. The child says the idea; a personalised storybook plays minutes later.	Voice-first input a child can drive; narrated video; a hero that stays the same on every page.
Educators in regional-language classrooms	A teacher builds reading material in the language of instruction — Hindi, Portuguese, Spanish — on school hardware.	Same-language story text *and* narration; printable A4 PDF with correct Devanagari/CJK typography; offline operation.
Regional-language content creators	YouTube/kids-content studios producing story videos outside English.	Subtitled MP4 with Ken Burns motion; style presets (watercolor / comic / 3D-animation look); per-page repaint; seed reproducibility.
Speech & language therapy	A therapist turns a child's own spoken sentences into material the child then reads back aloud.	The child's idea reflected back as printed and narrated text — motivation built from their own words, kept private on-device.
Libraries & reading programs	Story-creation kiosks at community events.	One GPU workstation, no per-use cost, no accounts, take-home PDF.

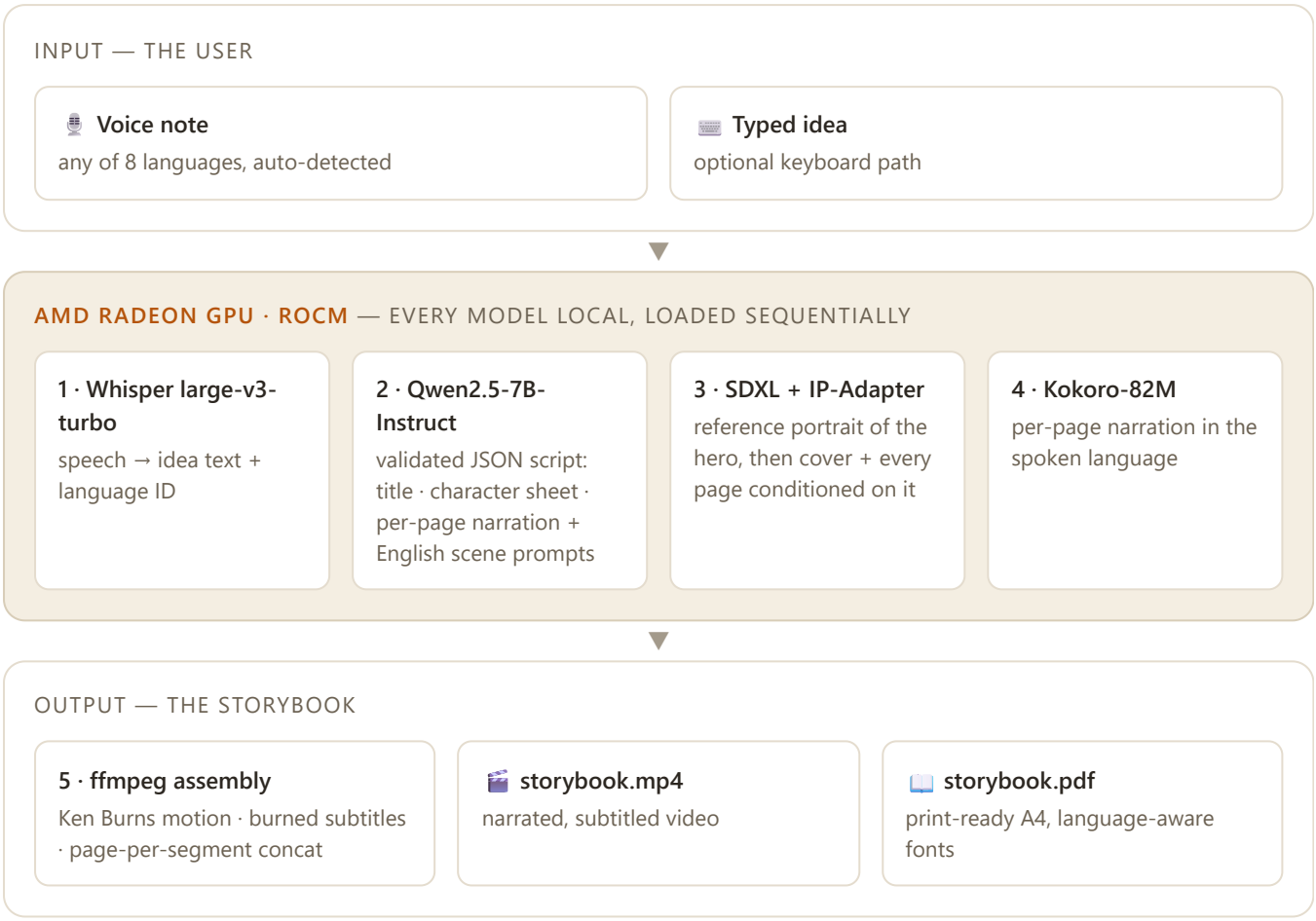
A concrete session

A parent opens Katha's web app, taps the microphone, and their daughter says *"एक हाथी जो उड़ना चाहता था"* ("an elephant who wanted to fly"). Katha detects Hindi, writes a six-page story with a named hero, paints a reference portrait, illustrates six watercolor pages holding that same elephant throughout, narrates each page in a Hindi voice, and assembles a subtitled video with gentle pan-and-zoom motion — plus a printable PDF for the bookshelf. If page four's painting isn't right, one click repaints just that page. Total time on a Radeon PRO W7900: under three minutes.

Deliverable forms

- **Web application** (Gradio): microphone capture, art-style and length controls, live stage progress, page gallery, per-page repaint, video player, PDF download.
- **CLI**: single command from a voice-note file or typed idea to finished outputs — scriptable for batch or kiosk use.
- **Artifacts per run**: `storybook.mp4`, `storybook.pdf`, page PNGs, narration WAVs, the structured story script (`story.json`), and a measured `benchmark.json`.

3 System Architecture



Sequential VRAM orchestration

The defining architectural decision: stages run **strictly sequentially, and each unloads its model before the next loads** (explicit teardown + `torch.cuda.empty_cache()` on the HIP backend). Peak memory is therefore the *largest single model*, not the sum — measured at **14.3 GB** during the Qwen stage. The same code that runs on a 48 GB workstation card runs unmodified on a 16 GB consumer Radeon; on smaller cards a single environment variable swaps in Qwen2.5-3B.

The story script is a typed contract

The LLM's output is not prose — it is JSON validated against a Pydantic schema (title, language, character name, an English-only *character sheet*, and per-page narration + scene prompt). A malformed response is fed back to the model with the validation error for one corrective retry. Downstream stages consume only the validated object, which is also persisted as `story.json` for reproducibility.

Failure containment

- IP-Adapter unavailable → automatic fallback to character-sheet + per-page-seed consistency, with a logged warning (`KATHA_IP_ADAPTER=0` forces it).
- Model hubs unreachable → endpoint fallback baked into setup and runners.

- Every run writes `benchmark.json` (per-stage wall time + peak VRAM measured on-GPU) — performance is evidence, not anecdote.

4 Model & Algorithm Introduction

ROLE	MODEL (ALL OPEN-WEIGHTS)	PRECISION / FOOTPRINT ON GPU
Ears	`openai/whisper-large-v3-turbo`	fp16 · ~1.6 GB · multilingual ASR + language ID
Storyteller	`Qwen/Qwen2.5-7B-Instruct`	fp16 · 14.3 GB peak · structured-JSON generation
Illustrator	`stabilityai/stable-diffusion-xl-base-1.0` + `h94/IP-Adapter` (SDXL)	fp16 · 9.7 GB peak · 1024^2 @ 30 steps · adapter scale 0.55
Narrator	`hexgrad/Kokoro-82M`	fp16 · 0.8 GB · 8 languages, 24 kHz
Bookbinder	ffmpeg (libx264/aac/libass) + PIL	CPU · zoompan Ken Burns, SRT burn-in, A4 PDF composition

Character consistency algorithm

The difference between "four random pictures" and "a book" is that the hero looks the same on every page. Katha layers three mechanisms:

- **Character sheet.** The LLM produces one detailed English physical description, injected verbatim into every illustration prompt.
- **Reference portrait + IP-Adapter.** Before any page is painted, Katha renders a portrait of the hero, then conditions the cover and all pages on that image (image-prompt adapter, scale 0.55) — anchoring identity at the feature level, not just the text level.
- **Deterministic seeds.** Page *n* uses `base_seed + n`; runs are reproducible and individual pages can be repainted without touching the rest.

Prompt budget engineering

SDXL's CLIP encoders truncate at 77 tokens — a failure we observed directly on-GPU (style suffixes silently chopped). Katha therefore treats the prompt as a budget: compact style presets, word-capped character sheet (≤18 words) and scene prompt (≤22 words), with caps enforced in code and mirrored in the LLM's instructions. Composed prompts measure ~48 words, safely inside the window.

Multilingual pipeline discipline

One rule keeps eight languages working: **narration in the speaker's language, scene prompts always in English.** Whisper detects the language; Qwen writes narration in it while emitting English prompts for SDXL (whose text encoders are English-trained); Kokoro selects a matching voice (e.g. `hf_alpha` for Hindi); the PDF

exporter selects Devanagari/CJK-capable Noto fonts. An unsupported language degrades gracefully to English narration with a logged warning.

5 Adaptation for AMD Radeon GPU / ROCm

Katha was developed and validated end-to-end on AMD hardware — Radeon Cloud instances with a Radeon PRO W7900 (gfx1100, 48 GB), ROCm 7.2.1, PyTorch 2.9.1 (HIP backend).

Measured performance (from a real run's `benchmark.json`)

STAGE	TIME	PEAK VRAM	NOTES
story — Qwen2.5-7B fp16	30.5 s	14.3 GB	~1,800-token structured script
illustrate — SDXL, 5 × 1024^2 @ 30 steps	60.5 s	9.7 GB	steady-state **~7 s per image** (4.07 it/s) after first-image kernel warm-up
narrate — Kokoro-82M	48.4 s	0.8 GB	title + 4 pages, 24 kHz
assemble — ffmpeg	17.4 s	—	CPU-side encode
Total, 4-page storybook	**≈ 2.6 min**	**14.3 GB**	models warm in cache

How Katha uses the Radeon well

- **fp16 everywhere.** All four neural stages run half-precision on the HIP backend — including SDXL attention through PyTorch SDPA (AOTriton efficient-attention path).
- **Peak-VRAM engineering.** Sequential load/unload turns a ~26 GB model portfolio into a 14.3 GB peak — the difference between "needs a workstation card" and "runs on a 16 GB consumer Radeon." `KATHA_LLM_MODEL=Qwen/Qwen2.5-3B-Instruct` drops the peak further for smaller cards.
- **Kernel behavior measured, not assumed.** First SDXL image absorbs one-time JIT warm-up (81 s observed, including AMD aiter kernel builds); subsequent images run ~7 s. The benchmark harness separates and reports this honestly.
- **MIOpen tuned for UX.** Kokoro's LSTM triggers verbose MIOpen workspace warnings; Katha ships with `MIOPEN_LOG_LEVEL=3` so user-facing logs stay clean while errors remain visible.

Deployment hardening on Radeon Cloud

Real deployments meet real networks. Katha's setup scripts are idempotent and region-aware: they activate the image's ROCm venv (`/opt/venv`), fall back to a Hugging Face mirror endpoint and a PyPI mirror when the defaults are unreachable, park the ~25 GB model cache and pip wheel cache on persistent storage so a rebuilt instance is productive in about a minute, and pre-install the one runtime dependency (spaCy's English model) whose default download path fails behind TLS-intercepting proxies. A fresh ROCm box goes from `git clone` to generating storybooks with two commands.

6 Deliverables, Status & Roadmap

Submission deliverables

ITEM	DETAIL
Source code	github.com/N-45div/Katha — MIT license; README with environment configuration, startup guide, and dependency list
Web application + CLI	`bash scripts/run_ui.sh` · `python cli.py idea.wav --style watercolor`
Demo video (3–5 min)	Recorded on a Radeon Cloud W7900 instance: CLI and web UI, English + Hindi storybooks, per-page repaint, `rocm-smi` visible
This profile document	`docs/PROJECT_PROFILE.pdf`
Slide deck	`docs/KATHA_DECK.pdf` — creative scenarios & practical value

Validation status

- **Validated on Radeon:** full five-stage pipeline, English storybook, measured benchmarks (Section 5).
- **Written and hardened, validated in the final demo session:** IP-Adapter conditioning (with automatic fallback), web UI, Hindi end-to-end, PDF export.

Roadmap beyond the hackathon

- **Story continuity:** multi-chapter books that remember previous adventures of the same hero.
- **Child-safe local guardrails:** a lightweight on-device content filter pass over story JSON.
- **More Indic languages:** Tamil, Telugu, Bengali narration as open TTS coverage grows.
- **ONNX/quantized variants:** push peak VRAM under 8 GB for entry-level Radeons and Ryzen AI iGPUs.

> **One sentence.** Katha turns a child's spoken idea into their own illustrated, narrated storybook in under three minutes — private by architecture, multilingual by default, and running entirely on an AMD Radeon GPU.